

# 61. Un tableau de « struct »

Trois ennemis, c'est UNE ligne de plus — pas neuf variables.

## 1. Ce qu'il faut comprendre

`troupe[i].vie` : l'index est calculé à l'exécution, le décalage du champ est connu à la compilation. **Le compilateur fait le calcul d'adresse** ; nous, nous écrivons ce que nous voulons dire.

C'est ici que la `struct` prend son sens. Passer de un à trois ennemis coûte une ligne ; sans elle, il aurait fallu six variables de plus et six lignes à tenir d'accord.

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1494 octets** de cartouche.

### 1. UN ENREGISTREMENT — ENNEMI

```
struct Ennemi {  
    uint8_t x, y, vie;  
};
```

### 2. UN TABLEAU — TROUPE

```
Ennemi troupe[3];
```

### 3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

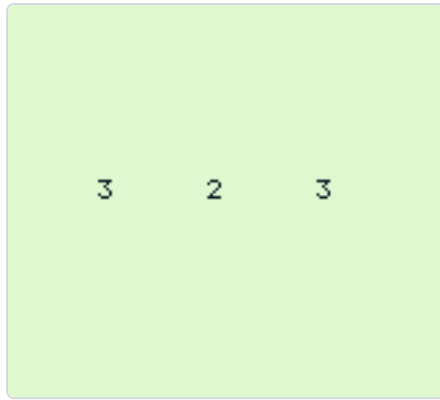
```
int main() {  
    for (uint8_t i = 0; i < 3; i++) {  
        troupe[i].x = 20 + i * 40;  
        troupe[i].y = 60;  
        troupe[i].vie = 3;  
    }  
  
    troupe[1].vie--;  
  
    for (uint8_t i = 0; i < 3; i++) nombre(4 + i * 5, 8, troupe[i].vie, 1);  
  
    while (true) {  
        image();  
    }  
}
```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

## 4. Ce qu'on doit voir

« 3 2 3 » — seul celui du milieu a été touché.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le premier est intact
- ✓ celui du milieu a perdu une vie
- ✓ le troisième est intact
- ✓ écrire dans une case n'a pas touché ses voisins

Ensuite : **62. Une méthode : la fonction qui connaît son objet** — niveau 8 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++